Insertion_Algorithm All_Questions

Question #01

//Simple just loops & Explained in DRY_RUN

(DRY_RUN)

```
void bubblesort(int ary[], int n)        n=4
{
  for(int i=0; i<n-1; i++)               0<3
  {
    bool swapped = false;
    for(int j=0; j<n-1; j++)             0<3
    {
      if(ary[j] > ary[j+1])              7>3
      {
        int t = ary[j]                   t=7
        ary[j] = ary[j+1]                ary[0] = ary[1]
                                         ary -> {3,3,6,2}
        ary[j+1] = t;                    ary -> {3,7,6,2}
        swapped = true;
      }
    }
    b again
    for(int j=0; j<---)                  1<2
    {
      if(---)
        swap ;                           ary -> {3,6,7,2}
```

Suppose: our ary is {7,3,6,2} n=4

⑤ again for(——) 2<3 ✓

{

if(——) 7>3 ✓

{

swap: arr → {2, 6, 3, 7

}

⑥ for(——) 3<3 ✗

if(!sw) ✗

again for(——) 1<3

{

⑥ for(int j=0——) 0<3 ✓

{

if(——) 2>6 ✗

⑥ again {for(——) 1<3 ✓

{

if(——) 6>3 ✓

{

swap; ——→ arr → {2, 3, 6, 7}

}

⑥ again for(——) 2<3 ✓

{

if(——) 6>7 ✗

if(!swap) ✗

↓

continue until loops end.

Question #3

// Explained in DR (DRY RUN)

suppose: our ary is; { 3, 2, 7, 6 } n = 4

```
void SS(int* ary, int n)
{
a. for(int i=0; i<n-1; i++)        0<3 ✓
   {
     int mi=i;                     mi = 0
b.   for(int j=i+1; j<n; j++)      1<4 ✓
     {
       if(ary[j]<ary[mi]) ✓        2<3
       {
         mi=j;                     mi=1
       }

b again: for(—) ✓                  j=2<4
     {
       if(—) ✗                     6<3
     }
     for(—) ✓                      j=3<4
     {
       if(—) ✗                     7<3
     }
   if(mi≠i){                       i=0, mi=1 -> swap
     int t=ary[i];
swap: } ⟶ ary -> {2,3,7,6}
```

Question # 03

(DRY_RUN)

suppose ary is : {3, 2, 7, 6}, n = 4

```
void inS(int ary[], int n)
{
   for(int i=1; i<n; i++)          1<4 ✓
   {
      int key = ary[i];    ary[i]=2
                                   key = 2
      int j = i-1;                 j = 0

      while(j>=0 && ary[j]>k) ✓    0>=0 && 3>2
      {
         ary[j+1] = ary[j];        -> ary = {3,3,7,6}
         j--;
      }
      whil(—) ✗                    j = -1+1 = 0
      ary[j+1] = key;              -> ary = {2,3,7,6}
   }

again for(——){        ✓  2<4
   int K = ary[i]          K=7
   int j = i-1;            j=1
   while(——)  ✗            3 ≯ 7

again for(——){             3<4
   int K = ary[i]          K=6
   int j = i-1;            j = 2
                       ✓   7>6
        ↓
     continue
```

Question # 04

* Explained in DRY RUN)

Suppose: Our ary is: {3, 2, 7, 6}
L→0 R→3

•) ⓐ void mergesort(int ary[], int L, int R)
{
if(L<R) ✓ 0<3
{
int mid = L+(R-L)/2; 0+(3) = 2
mergesort(ary, L, mid); mid=2

ⓑ mergesort(int* ary, int L, int R)
L→0 R→2
if(L<R) ✓ 0<2
{
int m = L+(R-L)/2; m=1
mergesort(ary, L, mid);

ⓒ mergesort(int* ary, int L, int R)
if(L<R) ✓ 0<1
{
int m = L+(R-L)/2; m=0
mergesort(ary, L, mid);

ⓓ ms(int ____)
if(L<R) ✗ 0<0

merges(ary, midl, right);

ⓓ ms(ary, 1 ____)
if(L<R) 1<2 ✓

merge(arr, L, mid, high);

0 0 1

void merge (int arr, int L, int M, int R)
{

int n1 = M - L + 1; n1 = 1
int n2 = R - M; n2 = 1

int LA[n1];
int RA[n2];

0<1 ← for (int i=0; i<n1; i++)
{
LA[0]={3} ← LA[i] = arr[L+i]
}

0<1 ← for (int i=0; i<n1; i++)
{
RA[0]={3} RA[i] = arr[mid+1+i]
}

int i=0;
int j=0;
K=0 ← int k=Left; K=0

✓ ← while(i<n1 && j<n2)
{
✗ if(——) ✗

arr[K] = LA[i]

✓ else
{
K
arr[0]=3 arr[k] = RA[j];
j=1 j++;
}
k++; }
k=1

0<1 ← while (i<n1)
{
arr[i]=3 ← arr[k]=LA[i]
i=1 i++;
k=2 k++;
}

10-10X ← while(j<n2)

return to (5)

continued.....

Question # 05

// Explained in DR:
(DRY _ RUN)

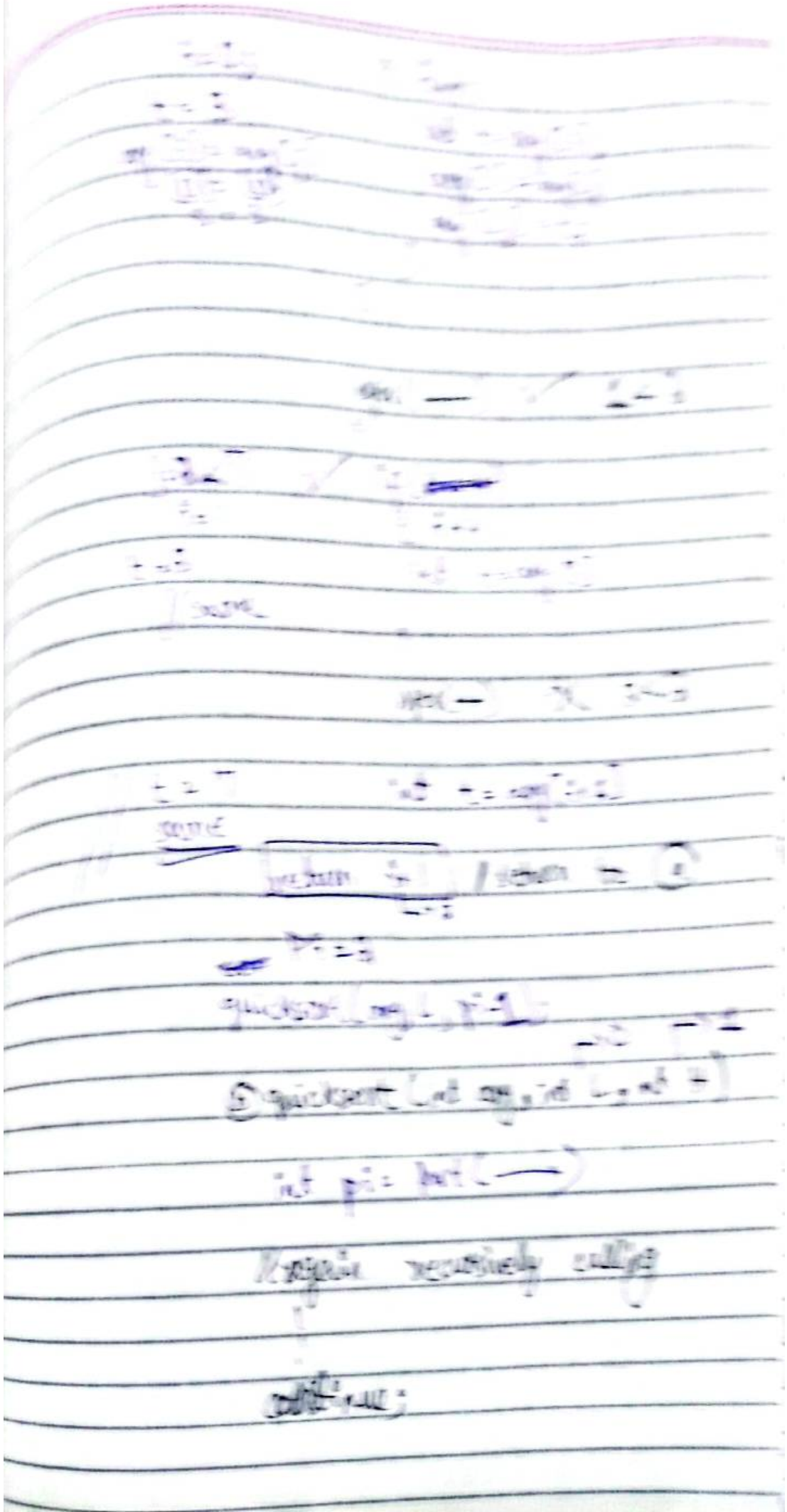
(a) void quicksort(int* arr, int L, int H) L→0, H→2
{
if(L<H) ✓ 0<H

Suppose arr is {3,2,7} size=3

int p=partition(arr, L, H);

b) partition(int* arr, int L, int H) L→0, H→2
{
piv=7 ← int piv=arr[high];
i=-1 ← int i=low-1;

0<2 ✓ for(int j=L; j<H; j++)
{
3<7 ✓ if(arr[j]<piv)



Question # 07

// Explained in DR: (DRY_RUN)

Suppose: arr ary is {3, 2, 7}
n = 3

```
void cs(int ary[], int n)
{
  int max = ary[0]          max = 3
  1) for(—) ✓
  {
    if(—) ✗                 2 > 3
    { max = ary[i]; }
  2) again: for(—) ✓
  {
    if(—) ✓                 7 > 3
    { max = ary[i]; }       max = 7
  }
  3) again for(—) ✗ 3 < 3 ✗
  int count[max+1] = {0};
  4) for(int i=0; i<n; i++)
  {
    count[ary[i]]++;        ary[0] = 3
  }                         count[3]++; {0,0,0,1,
  a) ag  count[2]++; {0,0,1,1,0,0,0,0}
  b) ag  count[7]++; {0,0,1,1,0,0,0,1}
```

```
for(i=0; i<=max; i++)
{
    if(count[i]>0)
    {
        prompt;          // every element =1
    }
}

for(int i=1; i<=max; i++)
{
    count[i] += count[i-1];      0+=0=0
}
```

arg: { 0, 0, 1, 1, 0, 0, 0, 1 }
arg: { 0, 0, 1, 2, 0, 0, 0, 1 }
arg: { 0, 0, 1, 2, 0, 0, 0, 1 }
arg { 0, 0, 1, 2, 2, 0, 0, 1 }
arg { 0, 0, 1, 2, 2, 2, 0, 1 }
arg { 0, 0, 1, 2, 2, 2, 2, 1 }
arg { 0, 0, 1, 2, 2, 2, 2, 3 }

```
int output[n];      output[3]

for(int i=n-1; i>=0; i--)
{
    output[count[arg[i]]-1] = arg[i]
        [2] = arg[i]   ⟶  output[3] = { _, _, 7 }
    count[arg[i]]--;   ⟶  count[7] = 2
}
```

arg: for(-) { output[0] = arg[2] → output[3] = { 2, _, 7 }
count[2]--; → count[2] = 0 (arg[i]=2, c[2]=1, 1-1)
arg for(-) { output[1] = arg[0] → output[3] = { 2, 3, 7 }
count[3]--; → count[3] = 1